IMPLEMENTATION OF WORD EMBEDDINGS FOR NLP USING WORD2VEC

Objective: Implement Word2Vec using CBOW and Skip-gram, demonstrate one-hot encoding and word-vector representations, and examine semantic similarity between words.

1. Execution Workflow

Stage	Description
Input text	Load customer reviews from sentiment_dataset.csv.
Preprocessing	Convert reviews to lowercase tokens and create a vocabulary.
One-hot encoding	Represent each vocabulary word as a sparse binary vector.
Word2Vec training	Train separate CBOW and Skip-gram models on the tokenized reviews.
Word vectors	Retrieve the learned dense vector for a selected word.
Semantic similarity	Find nearby words and calculate cosine similarity between word vectors.

2. Software Requirements

Use Python with Gensim, pandas and NumPy. Install the packages in Command Prompt or VS Code terminal:

```
pip install --upgrade pandas numpy gensim
```

Check the Gensim installation:

```
python -c "import gensim; print(gensim.__version__)"
```

3. Dataset Preparation

Use the previously created **sentiment_dataset.csv**. The file must contain a column named **Review**. The Sentiment column is not used to train Word2Vec because Word2Vec learns from word context without requiring class labels.

Review	Sentiment
The product is excellent	Positive
Amazing quality and fast delivery	Positive
Very poor quality	Negative
The service was terrible	Negative

Dataset limitation: The 100-review synthetic dataset is suitable for demonstrating the program. It is too small and repetitive to learn reliable general-purpose semantic representations.

4. Concepts Used in the Experiment

A. One-hot encoding

One-hot encoding represents each word using a binary vector. For the vocabulary **[excellent, is, product, the]**, each word receives a vector with one 1 and the remaining entries set to 0.

Word	excellent	is	product	the
excellent	1	0	0	0
is	0	1	0	0
product	0	0	1	0
the	0	0	0	1

Limitation: One-hot encoding does not express similarity between words. Each word has a separate position, and the vector length grows with the vocabulary.

B. Word embeddings

Word embeddings are dense numerical vectors learned from word-context patterns. Words that occur in similar contexts may acquire vectors that are close to one another. The vector values are learned by the model; they are not manually assigned.

C. CBOW (Continuous Bag of Words)

CBOW predicts a target word from its surrounding context words. For example, in **“The product is excellent”**, surrounding words can be used as context to predict **“excellent”** when it is the target.

D. Skip-gram

Skip-gram does the reverse: it uses a target word to predict nearby context words. For example, the target **“excellent”** is used to predict words that occur around it.

CBOW	Skip-gram
Context words predict the target word.	The target word predicts context words.
Often faster to train.	Can work well for rare words when enough training data is available.

E. Semantic similarity

Cosine similarity compares the direction of two word vectors. A value closer to 1 indicates similar vector directions; a value near 0 indicates little directional similarity. Negative values indicate vectors pointing in opposing directions.

Cosine similarity: $(A \cdot B) / (||A|| \times ||B||)$

5. Complete Python Implementation (Part 1)

Create a file named **word2vec_implementation.py** in the same folder as **sentiment_dataset.csv**. Add the following code.

```
# IMPLEMENTATION OF WORD EMBEDDINGS USING WORD2VEC

import pandas as pd
import numpy as np

from gensim.models import Word2Vec
from gensim.utils import simple_preprocess

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")

if "Review" not in data.columns:
    raise ValueError("CSV must contain a Review column.")

reviews = data["Review"].fillna("").astype(str)

# STEP 2: Text preprocessing and tokenization
sentences = [
    simple_preprocess(review, deacc=True, min_len=1)
    for review in reviews
]

# Remove empty tokenized reviews
sentences = [tokens for tokens in sentences if tokens]

if not sentences:
    raise ValueError("No usable review text was found.")

print("First tokenized sentence:")
print(sentences[0])

# STEP 3: Demonstrate one-hot encoding
sample_sentence = "the product is excellent"
words = sample_sentence.split()
vocabulary = sorted(set(words))

print("\\nVocabulary:")
print(vocabulary)

print("\\nOne-Hot Encoding:")
for word in vocabulary:
    vector = np.zeros(len(vocabulary), dtype=int)
    index = vocabulary.index(word)
    vector[index] = 1
    print(word, ":", vector)

# STEP 4: Train CBOW model
print("\\nTRAINING CBOW MODEL")

cbow_model = Word2Vec(
    sentences=sentences,
    vector_size=50,
    window=3,
    min_count=1,
    workers=1,
    sg=0,
    epochs=100,
    seed=42
)

print("CBOW training completed!")

# STEP 5: Train Skip-gram model
print("\\nTRAINING SKIP-GRAM MODEL")

skipgram_model = Word2Vec(
    sentences=sentences,
    vector_size=50,
    window=3,
    min_count=1,
    workers=1,
    sg=1,
    epochs=100,
    seed=42
)

print("Skip-gram training completed!")
```

5. Complete Python Implementation (Part 2)

Continue the same Python file with the following code:

```
# STEP 6: Display word vectors
word = "excellent"

if word in cbow_model.wv:
    print("\\nCBOW Vector for", word)
    print(cbow_model.wv[word])

    print("\\nSkip-gram Vector for", word)
    print(skipgram_model.wv[word])

    print("\\nVector Dimensions:")
    print("CBOW:", cbow_model.wv[word].shape)
    print("Skip-gram:", skipgram_model.wv[word].shape)
else:
    print("The word is not present in the trained vocabulary.")

# STEP 7: Find similar words
print("\\nCBOW - MOST SIMILAR WORDS")
if word in cbow_model.wv:
    print(cbow_model.wv.most_similar(word, topn=5))

print("\\nSKIP-GRAM - MOST SIMILAR WORDS")
if word in skipgram_model.wv:
    print(skipgram_model.wv.most_similar(word, topn=5))

# STEP 8: Calculate semantic similarity
word_pairs = [
    ("excellent", "amazing"),
    ("poor", "terrible"),
    ("product", "quality")
]

print("\\nSEMANTIC SIMILARITY USING CBOW")
for w1, w2 in word_pairs:
    if w1 in cbow_model.wv and w2 in cbow_model.wv:
        score = cbow_model.wv.similarity(w1, w2)
        print(w1, "<->", w2, ":", round(float(score), 4))
    else:
        print(w1, "<->", w2, ": one or both words are absent")

print("\\nSEMANTIC SIMILARITY USING SKIP-GRAM")
for w1, w2 in word_pairs:
    if w1 in skipgram_model.wv and w2 in skipgram_model.wv:
        score = skipgram_model.wv.similarity(w1, w2)
        print(w1, "<->", w2, ":", round(float(score), 4))
    else:
        print(w1, "<->", w2, ": one or both words are absent")

# STEP 9: Save both models
cbow_model.save("cbow_model.model")
skipgram_model.save("skipgram_model.model")

print("\\nBoth Word2Vec models saved successfully!")
```

6. How to Execute the Program

1. Create a folder named **Word2Vec_NLP**.

2. Place **sentiment_dataset.csv** and **word2vec_implementation.py** inside the folder.

3. Open Command Prompt or the VS Code terminal.

4. Navigate to the folder using the cd command.

5. Install pandas, NumPy and Gensim.

6. Run the Python file and observe the tokenization, vectors and similarity results.

Terminal commands

```
cd path\to\Word2Vec_NLP
pip install --upgrade pandas numpy gensim
python word2vec_implementation.py
```

7. Important Word2Vec Parameters

Parameter	Explanation
vector_size=50	Each word is represented using a vector of 50 numerical values.
window=3	Uses nearby words within a context window of up to 3 tokens on either side.
min_count=1	Includes words that appear at least once.
sg=0	Selects CBOW.
sg=1	Selects Skip-gram.
epochs=100	Trains the model for up to 100 passes over the corpus.
workers=1	Uses one worker thread; useful for more repeatable training.

8. Understanding the Expected Output

Output	What it shows
Tokenized sentence	Words converted into a list of tokens.
One-hot vectors	Binary vectors with one active position per word.
Word vector	A dense vector learned for a word by CBOW or Skip-gram.
Most similar words	Words with the nearest vector directions according to cosine similarity.
Similarity score	Cosine similarity for a selected pair of words.
Saved models	cbow_model.model and skipgram_model.model.

Important: Vector values and similarity scores are produced by training and vary with the corpus and training settings. Do not expect illustrative values to match your output.

9. CBOW and Skip-gram Comparison

CBOW	Skip-gram
Predicts a target word from its context.	Predicts context words from a target word.
Often computationally faster.	Can learn useful representations for rare words when trained on enough data.
Uses surrounding context as input.	Uses the centre/target word as input.

10. Suggested Laboratory Experiment

Title: Implementation of Word Embeddings for NLP using Word2Vec.

Aim: To implement Word2Vec using CBOW and Skip-gram models, demonstrate one-hot encoding and word-vector representations, and analyse semantic similarity between words.

Learning Outcomes

1. Understand one-hot encoding and word embeddings.
2. Implement CBOW and Skip-gram models using Gensim.
3. Generate word-vector representations.
4. Calculate semantic similarity using cosine similarity.
5. Compare the CBOW and Skip-gram approaches.

11. Teaching Recommendations

First demonstrate how a sentence is tokenized. Next, show one-hot vectors and explain why they do not capture semantic similarity. Then train CBOW and Skip-gram, inspect a word vector, and calculate similarity for word pairs. Emphasize that Word2Vec learns from word contexts and does not use the Positive/Negative labels in this experiment.

Dataset recommendation: The 100-review synthetic dataset is only a starting point. For more meaningful embeddings, use a larger and more varied text corpus. Words occurring only once may have poorly estimated vectors, even when min_count=1 includes them.

12. Reference

Gensim Word2Vec documentation: https://radimrehurek.com/gensim/models/word2vec.html